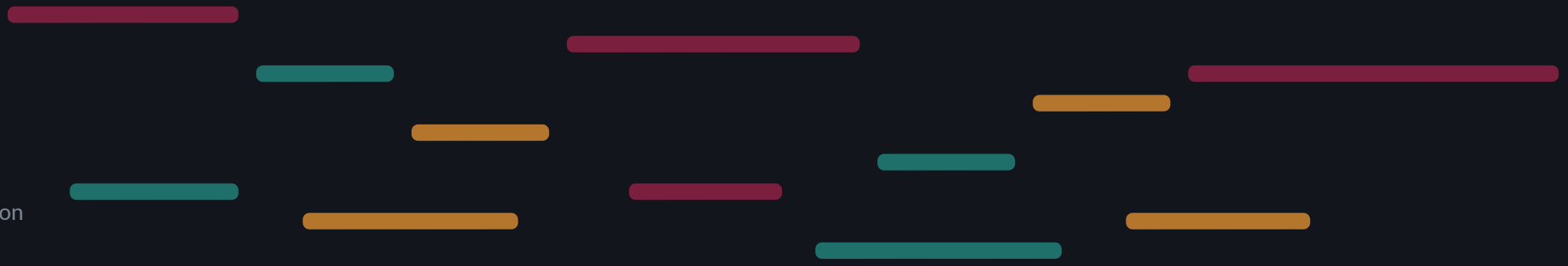
CADENCE

A music DSL that compiles to MIDI

A complete compiler — scanner, parser, semantic analyser, optimiser, verifier, virtual machine, code generator — written from scratch in pure Python.
No parser generator. No MIDI library. No third-party packages at all.



Compiler Design — project presentation

The idea in one slide

You write this. The compiler produces this.

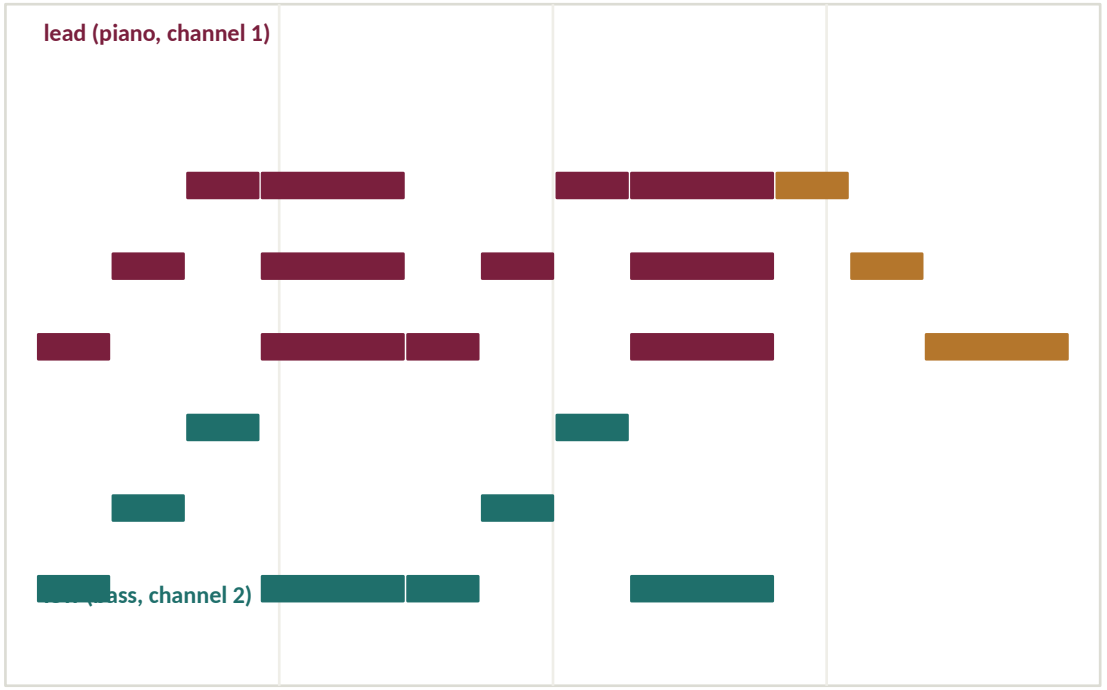
```
tempo 120 ;
key D minor ;
duration 8 ;
instrument piano = "grand piano" ;
instrument bass  = "electric bass" ;

phrase riff(t) =
    transpose t of [ D4 F4 A4 <D4 F4 A4>/4 ] ;
phrase tail = [ ^5 ^3 ^1/4 r/8 ] ;

voice lead on piano = repeat 2 of riff(0) tail ;
voice low  on bass  = transpose -12 of
                      [ repeat 2 of riff(0) ] ;

play lead | low ;
```

17 lines of Cadence · examples/03_riff.cad



27 notes · 2 tracks · 3.75 seconds · a 348-byte .mid file that plays

93	83	59%	0
tokens scanned	IR instructions	less work after optimising	dependencies

Every module of the course, doing real work

Not a demo per module — one program that needs all of them at once.

Module	Syllabus topic	What Cadence actually does
1	Lexical analysis, regex → DFA, Lex	Direct method (nullable/firstpos/lastpos/followpos), then minimisation. 47 states. Python's re module is never used.
2	Ambiguity, LL(1), SLR, CLR, LALR	All four tables built from one grammar. LALR parses; recursive descent cross-checks it.
3	SDD, evaluation order, L-attributed	6 inherited + 3 synthesised attributes. A scale degree cannot be resolved without the inherited key.
4	Three-address code, backpatching, procedures	Quadruples, one procedure per phrase with an activation record, real backpatch lists for loop exits.
5	Basic blocks, DAG, data flow, peephole	13 passes. Live variables, reaching definitions, available expressions, dominators, natural loops.
5	Naive code generator for a VM + security checking	A VM with a frame stack, and a verifier that checks 5 safety properties by abstract interpretation.
6	Target machine, register allocation, activation records	MIDI channels are the registers: interference graph, Chaitin colouring, spilling.
7	DSL compilation, parallelism, SSA	Cadence IS the DSL. Parallel voices become independent tracks; SSA is the stated next step.

The pipeline

Twelve stages. Every intermediate artefact can be printed.

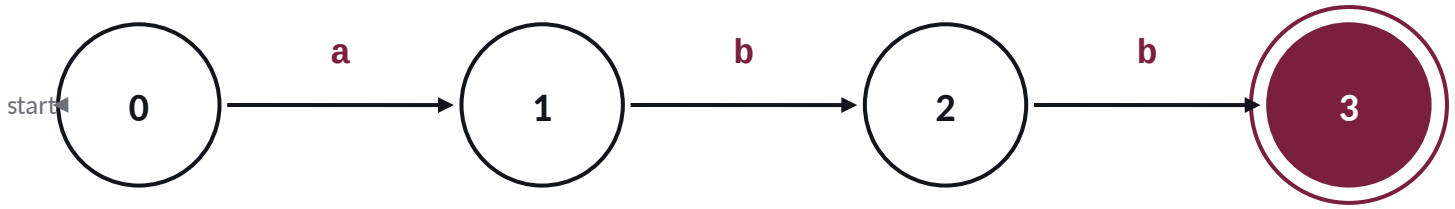


1 lexical analysis	regex → DFA → minimise → maximal munch
2 syntax analysis	LALR(1) table + recursive-descent cross-check
3 semantic analysis	L-attributed walk, symbol table, 12 checks
4 intermediate code	quadruples, backpatching, activation records
5 verification	5 safety properties by abstract interpretation
5 optimisation	13 passes, each one measured and proved inaudible
6 code generation	virtual machine, then graph-colouring allocation
— back end	Standard MIDI File, written byte by byte

Every arrow is a --dump flag: --dump tokens, ast, ir, passes, dataflow, channels, events.

Module 1 — regular expressions become a DFA directly

No NFA, no subset construction of an NFA, and no use of Python's re module.



state 0 loops on b · states 1, 2 and 3 all go back to state 1 on a · state 3 returns to 0 on b

```
TOKEN_RULES = [ ... ]            # 25 extended regular expressions
dfa = build_dfa(TOKEN_RULES)     # one tree, positions numbered, followpos
dfa = minimise(dfa)              # Moore partition refinement
```

Checked against the textbook example: build_dfa([("A", "(a|b)*abb")]) reproduces exactly this 4-state automaton, with 6 numbered positions.

25
token rules

101
positions

37
character classes

47
DFA states

Why 37 classes and not 256?

Two characters that appear in exactly the same set of positions can never be told apart by this automaton, so transitions are computed once per group. [0-9] is one column of the table, not ten.

Module 1 — the two rules that make a scanner a scanner

Longest match first; declaration order only breaks ties.

```
REST      (r|~)(/(1|2|4|8|16|32|64)\.?)?
PITCHNAME [A-G](#|b)?
NOTE      [A-G](#|b)?[0-8](/...)?
ID        [a-z_][a-zA-Z0-9_]*
```

four of the twenty-five rules, in priority order

input	rules that match	chosen	why
riff	ID (4 chars), REST (1 char)	ID	maximal munch: longest wins
r	REST (1), ID (1)	REST	tie → the rule declared first
D4/8	NOTE (4), PITCHNAME (1)	NOTE	maximal munch
D	PITCHNAME (1), NOTE (0)	PITCHNAME	NOTE needs an octave digit
tempo	ID (5)	TEMPO	keywords are folded after scanning

Keywords are not in the DFA

Twenty reserved words would add twenty near-identical paths to the automaton. Instead ID is scanned, then looked up in a keyword table — the same trick Lex users apply by hand. 'tempo' becomes TEMPO; 'tempoo' stays an ID.

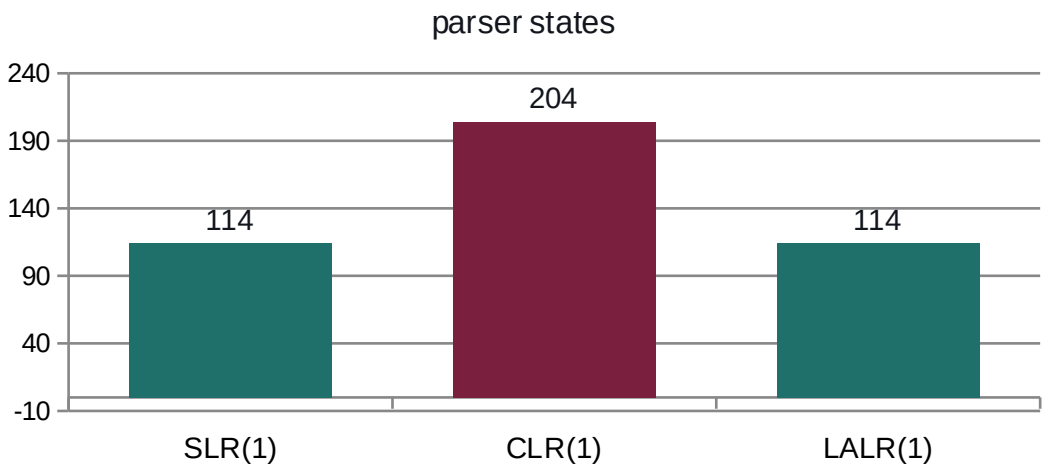
Errors do not stop the scan

```
[lexical error] line 7:20: unterminated string
    instrument piano = "grand piano ;
                       ^
    hint: add the closing "
[lexical error] line 8:31: stray character '$'
    voice tune on piano = [ C4 E4 $ G4 ] ;
                                  ^
```

panic-mode recovery: skip one character, keep scanning, report both

Module 2 — one grammar, four parsers, one honest comparison

56 productions. Built at start-up, in about 250 milliseconds.



parser	states	entries	s/r	r/r
SLR(1)	114	742	0	0
CLR(1)	204	1343	0	0
LALR(1)	**114**	**742**	0	0

The trade-off LALR exists to make — measured

CLR(1) keeps a separate state for every lookahead context: 204 states.
Merge the states that share a core and you are back to 114 — the same size as SLR(1) — without reintroducing a single conflict.
44 % fewer states than CLR, all of CLR's power on this grammar.

And LL(1)? It fails — which is the point.

- 56 conflicting entries in the LL(1) table.
- 8 nonterminals are left-recursive: arglist, comp, decls, expr, idlist, notelist, seq, term.
- So the grammar keeps its natural left recursion and is parsed bottom-up. Recursive descent survives only as the cross-check, with its left recursion hand-converted into loops.

Module 2 — the first grammar I wrote was ambiguous

The table builder found it. The fix was to change the language, not to hack the table.

first draft

```
comp -> comp comp          | seq
elem -> LPAREN comp RPAREN            // group
elem -> LPAREN notelist RPAREN SLASH dur
                                      // chord
```

after

```
comp -> comp BAR seq       | seq
seq  -> seq elem           | elem
elem -> LBRACK comp RBRACK            // group
elem -> LT notelist GT SLASH dur      // chord
```

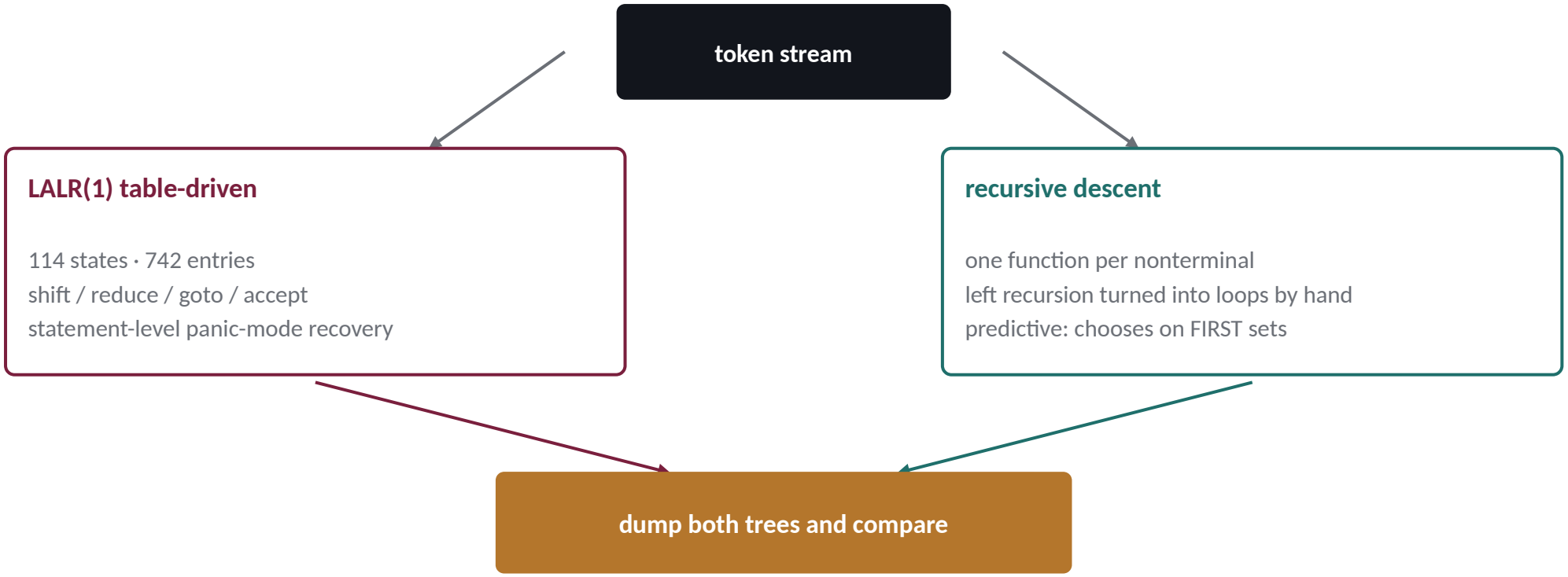
	states	shift/reduce	reduce/reduce	verdict
ambiguous draft	28	24	2	unusable
final grammar	114	0	0	parses

The reduce/reduce pair is the interesting one

After reading (D4 the parser cannot know whether it is inside a chord or inside a group — the decision needs the token after the closing bracket, which is more lookahead than LR(1) has. Chords now use angle brackets, so one token of lookahead is enough. Both grammars are still in the source, and the report prints the conflicts with the state, the lookahead and the two competing actions.

Module 2 — two parsers, one grammar, byte-identical trees

Differential testing: the cheapest strong evidence that a parser is correct.

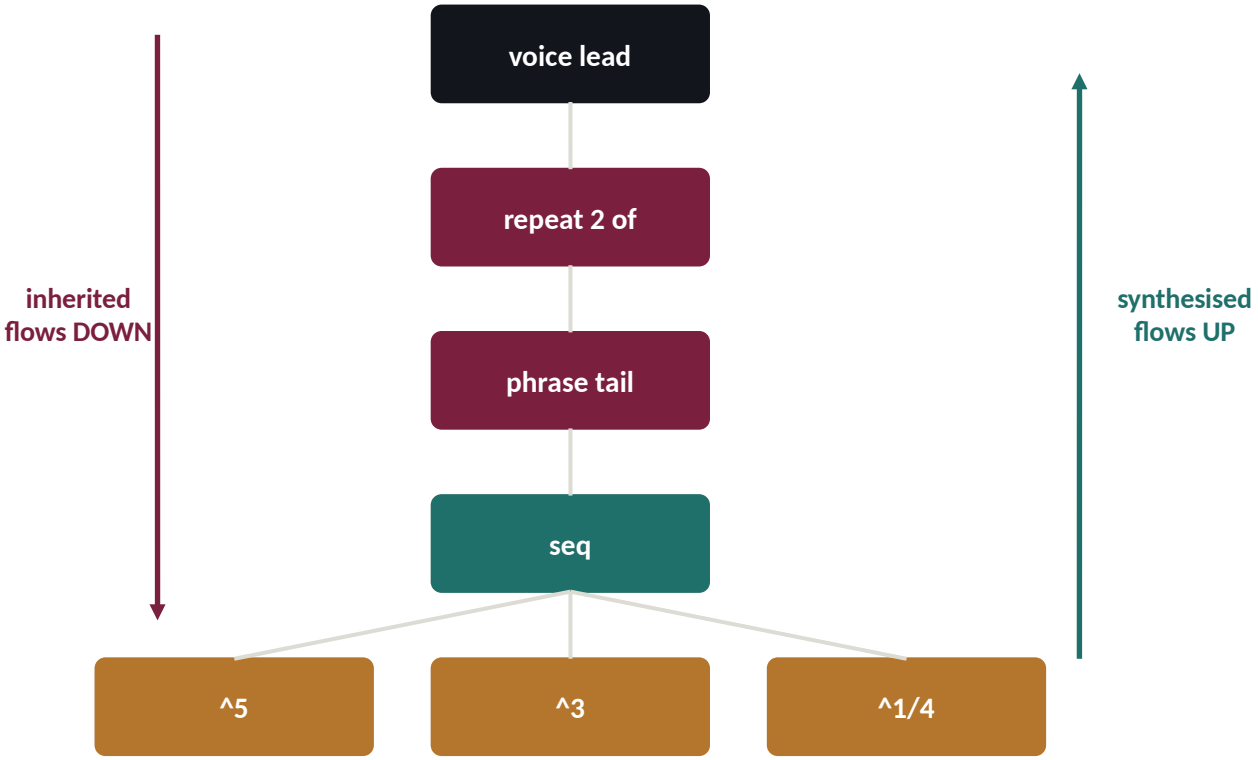



```
TREES IDENTICAL: True
```

Checked for all seven working examples on every run, and asserted in the test suite. If the two parsers ever disagree, the build says so — one of them has a bug, and I know immediately.

Module 3 — why this language genuinely needs inherited attributes

^5 is not a note. It becomes one only when the key reaches it.



down: key, mode, octave, duration, velocity, transposition, stretch

up: ticks, note count, pitch range

```
key D minor ;        // inherited: pc=2, mode=minor
^5   -> degree 5 of D minor
     -> scale [0,2,3,5,7,8,10]
     -> pc 2 + 7 = 9, octave 4
     -> MIDI 69           A4

change one word to 'D major' and every
note in the file changes -- no note edited
```

checked here	example message
undeclared / duplicate	'plano' is not a declared instrument
wrong arity	phrase 'riff' takes 1 argument but got 2
recursive phrase	phrase 'loop' is recursive (loop -> loop)
unknown instrument	did you mean: 'grand piano', 'piano'?
range checks	pitch 198 is outside MIDI range 0-127

Diagnostics a person can actually read

One deliberately broken example per phase ships with the project.

```
// bad_semantic.cad -- every line is legal syntax
tempo 900 ;
instrument piano = "grand pianoo" ;
phrase riff(t) = transpose t of [ C4 E4 ] ;
phrase loop    = [ C4 loop ] ;
voice tune on plano = riff(1, 2) ;
voice high on piano = transpose 90 of [ C8 ] ;
```

The grammar is perfectly happy with this file. Only the analyser can object.

```
$ python -m cadence examples/bad_semantic.cad
5 error(s), 2 warning(s)
-- no MIDI file written

$ python -m unittest discover -s tests
Ran 72 tests ... OK
```

```
[semantic error] line 12:1:
  tempo 900 is outside the playable range 20-400
    tempo 900 ;
    ^

[semantic error] line 14:1:
  unknown instrument name 'grand pianoo'
    instrument piano = "grand pianoo" ;
    ^
    hint: did you mean: 'grand piano', 'piano'?

[semantic error] line 17:1:
  phrase 'loop' is recursive (loop -> loop)
    hint: a phrase cannot play itself

[semantic error] line 19:23:
  phrase 'riff' takes 1 argument(s) but got 2
    voice tune on plano = riff(1, 2) ;
                          ^
    hint: declared on line 16 as riff(t)

[warning] line 20:41:
  pitch 198 is outside MIDI range 0-127
```

Module 4 — three-address code, and real backpatching

The generator is deliberately naive. That is what gives the optimiser work to do.

```
voice lead = repeat 2 of riff(0) ...

  0   t1 = 2
  1  L1:
  2   if t1 <= 0 goto ____   <-- target unknown
  3   PARAM 0                    when emitted
  4   CALL phrase riff, 4
  5   t1 = t1 - 1
  6   goto L1
  7  L2:                     <-- now it is known
```

makelist → merge → backpatch

The exit jump is emitted with an empty target and its index is kept on a list. When label L2 is finally generated, every jump on the list is patched. The textbook technique, in irgen.py.

```
every note, however simple, emits six:

  t2 = 62 + $off      // transposition
  NOTEON t2, vel 96
  t3 = 240 * $sn      // stretch numerator
  t4 = t3 / $sd       // stretch denominator
  DELAY t4
  NOTEOFF t2

even when $off = 0 and $sn/$sd = 1/1
```

Why write code you know is bad?

Because it is the standard front-end contract: generate uniformly, optimise later. It also means the before/after numbers on the next slides are real measurements of my own optimiser, not a rigged demonstration.

9 → 5
procedures

173
instructions, naive

2
deepest frame stack

quadruples
op, arg1, arg2, result

Module 5 — thirteen passes, and the DAG they search

Common subexpressions are found by value-numbering a basic block into a DAG.

1. CSE via DAG
2. inline phrases
3. constant folding
4. copy propagation
5. loop unrolling
6. CSE again
7. folding again
8. copy propagation
9. strength reduction
10. dead code elimination
11. drop unused procedures
12. peephole

Passes are re-run because each one exposes work for the others: inlining creates constants, folding creates copies, copy propagation creates dead code.

```
block B0 of phrase riff, as a DAG

  n1 = $off        n2 = 62
  n3 = ADD(n2, n1)  -> t2       D4
  n4 = MUL(240, $sn)-> t3
  n5 = DIV(n4, $sd) -> t4
  ...
  t6 is an alias of n4   <-- same
  t9 is an alias of n4       node

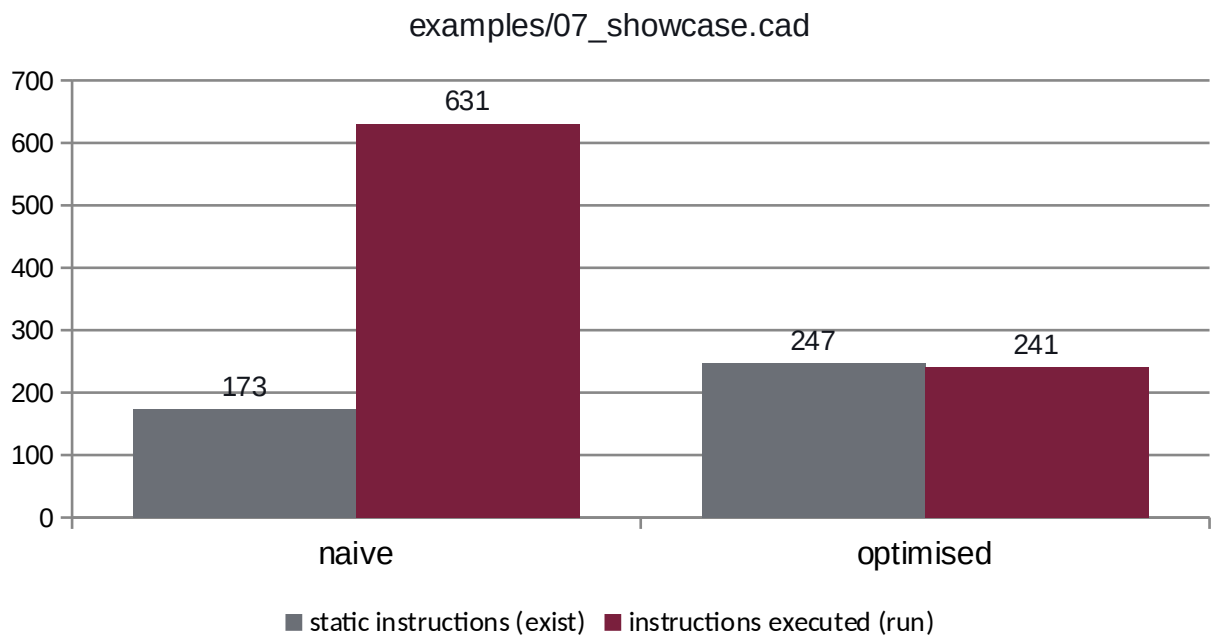
15 repeated expressions reused
```

A real bug, found by measuring

My first CSE pass reported zero eliminations on a block that obviously repeated 240 * $sn three times. The cause: I inserted the new value into the available table and then invalidated everything defined by that same instruction — killing the entry I had just created. Killing before inserting fixed it, and the pass immediately found 15 reuses.
Without a per-pass measurement table I would never have noticed.

Module 5 — the measurement that actually matters

Instructions that exist versus instructions that run. Optimisation trades one for the other.



+42.8 % bigger, 61.8 % less work

Inlining and unrolling deliberately add instructions so that the program executes fewer of them. Reporting only the static count would have hidden the win; reporting only the dynamic count would have hidden the cost.

pass	static	executed	music unchanged
CSE via DAG	173 → 173	631	yes
inline phrases	173 → 276	542	yes
constant folding	276 → 276	542	yes
copy propagation	276 → 276	542	yes
loop unrolling	276 → 567	462	yes
dead code elimination	567 → 325	241	yes
drop unused procedures	325 → 247	241	yes

The invariant the design is built on

After every pass the virtual machine re-runs the program and the note-by-note event stream is compared with the unoptimised run. A pass that changes what you hear fails the build. Checked for every example, at every level, on every run.

Module 5 — data-flow analysis, iteration by iteration

Basic blocks, dominators, back edges, and three classic fixed points.

```
B0  t1 = 2                  --> B1

B1  L1:                     --> B3, B2
    if t1 <= 0 goto L2

B2  PARAM 0 ... CALL riff
    t1 = t1 - 1
    goto L1                 --> B1  [back edge]

B3  L2:  RET

dominators   B0:{B0} B1:{B0,B1}
             B2:{B0,B1,B2} B3:{B0,B1,B3}
natural loop header B1, tail B2, body {B1,B2}
```

A back edge whose head dominates its tail is a natural loop — which is how the unroller knows what it may unroll.

analysis	direction	meet	used for
live variables	backward	union	dead-code elimination
reaching definitions	forward	union	copy propagation
available expressions	forward	intersection	common subexpressions
dominators	forward	intersection	finding loops
definitely assigned	forward	intersection	the code verifier

```
live variables, every iteration recorded

iteration 1   B0 IN {}      OUT {t1}
              B1 IN {t1}    OUT {t1}
              B2 IN {t1}    OUT {t1}
iteration 2   (unchanged)
iteration 3   (unchanged)  <-- fixed point

the report shows all three rounds, not just
the answer -- because the iteration IS the topic
```

A test asserts that the last two iterations are identical. That is what "fixed point" means.

Module 6 — activation records, for real

A phrase call pushes a frame. The VM shows you the stack while it runs.

frame layout for phrase riff(t)

return address	0
control link	1
$off (transposition)	2
$sn (stretch numerator)	3
$sd (stretch denominator)	4
$t (user parameter)	5
locals: t1, t2, t3 ...	6+

frame size 9 words · computed by irgen.frame_layout()

```
the stack, printed while the program runs

step 8
  voice lead [ret=-1, link=-1] {t1=2}
    phrase riff [ret=8, link=0] {$off=0,
                 $sn=1, $sd=1, $t=0}

step 49
  voice lead [ret=-1, link=-1] {t1=1}
    phrase riff [ret=8, link=0] {...}

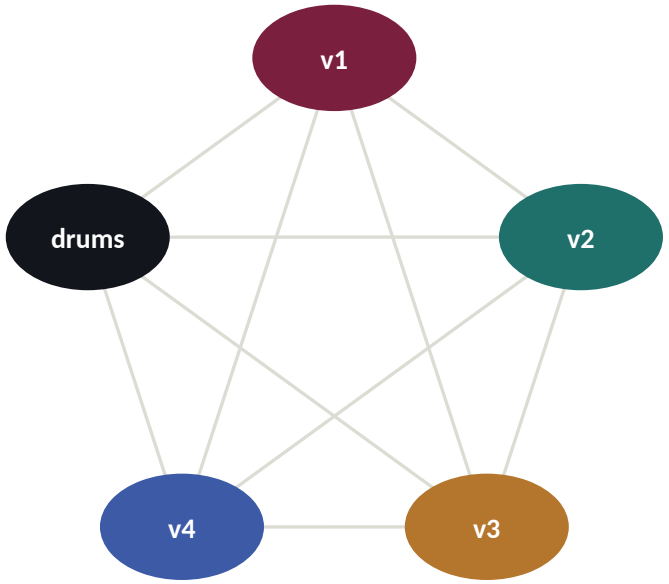
step 90
  voice lead [ret=-1, link=-1] {t1=0}
    phrase tail [ret=15, link=0] {...}
```

The VM is sandboxed

4 000 000 instructions, 64 frames, 200 000 events. A program that does not terminate is stopped and reported as an error instead of hanging the tool — which matters, because 'repeat n of' with a computed n is easy to get wrong. There is a test that an infinite loop is caught.

Module 6 — register allocation, where the registers are MIDI channels

Sixteen channels. Channel 10 is reserved for percussion. Colour the graph or spill.



an edge means the two voices sound at the same time, so they cannot share a channel

```
live ranges are half-open: a voice that
stops exactly when another starts does
NOT need a second channel
```

step	what happens
1 live ranges	first note to last note, per voice
2 interference	an edge for every overlapping pair
3 pre-colour	percussion is forced onto channel 10 (index 9)
4 simplify	repeatedly remove a node with degree < 15 (Chaitin)
5 select	colour in reverse order of removal
6 spill	if nothing can be removed, spill the voice with fewest notes

17
voices asked for

15
channels available

2
spilled

examples/06_spill.cad asks for seventeen simultaneous voices. The allocator spills the two with the fewest notes — they share a channel and may cut each other off, which the compiler warns about. This is exactly what a register allocator does when it runs out of registers, and it is audible.

The syllabus line most projects skip

“Implementation of a naive code generator for a virtual machine, security checking of virtual machine code.”

1	**every jump target exists**	no branch can leave the code array
2	**call arity and PARAM balance**	enough arguments pushed on every path
3	**parallel blocks properly nested**	depth agrees at every merge point
4	**no local read before it is written**	forward must-analysis, meet = intersection
5	**every procedure ends in RET**	control cannot fall off the end

```
$ ... --dump verify
  [pass] jump targets resolve         phrase riff
  [pass] call arity and PARAM balance phrase riff
```

Same idea, shipping systems

- JVM bytecode verifier — stack depth and type agreement at merge points
- WebAssembly validation — structured control flow, in-range branches
- eBPF verifier — bounded loops, definitely-initialised registers

run twice

once on the naive code, once on the optimised code — because an optimiser is a program too, and it can break invariants.

The tests hand-craft illegal code for all five, and check each one is caught.

The back end: no library, just bytes

Standard MIDI File format 1, assembled by hand and read back by an independent parser.

```
0000  4d 54 68 64 00 00 00 06   MThd....
0008  00 01 00 03 01 e0         format 1, 3 tracks, 480 tpq
000e  4d 54 72 6b 00 00 00 13   MTrk (tempo track)
0016  00 ff 51 03 07 a1 20      set tempo = 500000 us/quarter
001d  00 ff 58 04 04 02 18 08   time signature 4/4
0025  00 ff 2f 00               end of track
0029  4d 54 72 6b ...           MTrk (voice lead)
      00 c0 00                  program change: piano
      00 90 3e 60               note on  D4 velocity 96
      81 70 80 3e 40            delta 240, note off D4
```

81 70 is a variable-length quantity: seven bits per byte, high bit means “more to come”. 0x81,0x70 = 240 ticks.

written by hand	why it matters
header + track chunks	the file format is part of code generation
variable-length delta times	a real encoding problem, solved in 6 lines
one track per voice	matches the channel allocation exactly
tempo and time signature meta	so any player reproduces the right speed

348
bytes

27
notes

3
tracks

Read back to prove it

The test suite contains a MIDI parser that was written separately from the writer. It walks every chunk of every generated file and checks the format, the division, that every note that starts also stops, and that the count matches what the compiler reported. Byte-for-byte deterministic across runs.

How it is built, and how you run it

One folder, no installation, and a window that shows every phase.

	count	
Python modules	20	one per phase, plus the driver and the report
automated tests	72	all passing, stdlib unittest only
example programs	10	7 that work, 3 broken on purpose
third-party packages	0	no parser generator, no MIDI library
lines of C, C++ or C#	0	pure Python 3.10+

```
run.bat                     double-click, on Windows

python -m cadence song.cad
python -m cadence song.cad --report r.html
python -m cadence song.cad --dump all
python -m cadence song.cad -O0     # no optimising
python -m cadence --gui
python -m cadence --all
python -m unittest discover -s tests -v
```

Three ways to look inside

the window

seventeen tabs — tokens, DFA, grammar, parse trace, tree, symbols, attributes, IR, passes, CFG, data flow, frames, channels, piano roll

the HTML report

one self-contained file, no internet, every table the compiler built, plus an SVG piano roll — this is where the report screenshots come from

the terminal

--dump prints any phase, so everything can be diffed, piped and version-controlled

Every number, in one place

All of these are generated by the compiler while compiling examples/07_showcase.cad.

phase	measurement	value
1 lexical	token rules → positions → character classes → DFA states	25 → 101 → 37 → 47
1 lexical	tokens scanned	168
2 syntax	productions in the grammar	56
2 syntax	SLR(1) / CLR(1) / LALR(1) states	114 / 204 / 114
2 syntax	LALR(1) conflicts	0
2 syntax	conflicts in the ambiguous first draft	24 s/r + 2 r/r
2 syntax	shifts / reductions on this program	168 / 197
3 semantic	symbols in the table · phrases · voices	14 · 4 · 4
4 intermediate	procedures · instructions (naive)	9 · 173
5 optimisation	passes run · static instructions after	13 · 247
5 optimisation	instructions executed, before → after	631 → 241 (61.8 % saved)
5 optimisation	music changed by any pass	no
6 back end	voices · channels · spills	4 · 4 · 0
6 back end	notes · seconds · file size	83 · 9.6 · 891 bytes
whole run	compile time	85 ms

What I would build next

SSA form with phi-node insertion

module 7 — dominance frontiers are already computed, so this is the natural next step

textual LLVM IR output

emit a .ll file as plain text; no toolchain needed to show the connection to module 1's LLVM material

instruction scheduling

reorder independent NOTEONs within a chord; a scheduling problem with an audible result

a second back end

the same IR to WAV by synthesising samples, which would prove the IR is genuinely target-independent

---

Cadence

compiles music · pure Python · zero dependencies · 72 tests

thank you — questions welcome